PRESENTATION

BY GANG X7

C# | ENTITY FRAMEWORK CORE | SQLITE

TEAM MEMBERS

1. Mahmoud Aboshark
2. Mohamed Azab
3. Ahmed Ehab
4. Sohaila Hussam
5. Ragda Gad

HOW CONTACT BOOK WORKS

```
var contact = new Contact
{
    FirstName = firstName,
    LastName = lastName,
    PhoneNumber = phone,
    Email = email,
    Category = category,
    Address = address,
    IsDeleted = false
};

if (Save(() => _db.Contacts.Add(contact)))
{
    WriteLineCentered("Contact saved successfully."
}
Console.ResetColor();
```

Creating and Saving a Contact
Creates a new Contact object and assigns the user's input to its properties.

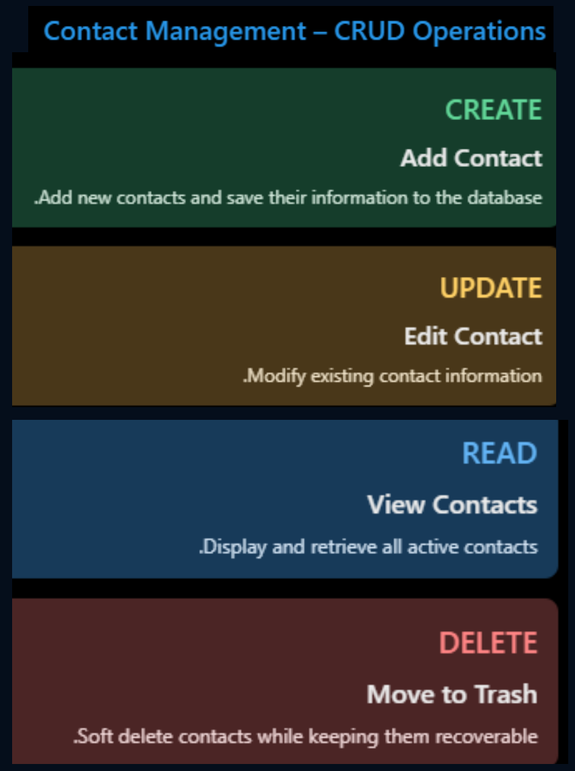
Main Steps:

- Create: Initialize a new Contact object.
- Assign: Store contact details.
- Set Status: Set IsDeleted = false.
- Save: Add the contact to the database using EF Core.
- Confirm: Display a success message.

Key Concepts: Object Initialization · EF Core · Database Persistence

CONTACT MANAGEMENT - CRUD OPERATIONS

```
private void UpdateContact()
{
    Console.ForegroundColor = ConsoleColor.Blue;
    WriteLineCentered("--- Update contact ---");
    WriteLineCentered("Tip: Type 'b' at any prompt to cancel/go back.");
    WriteLineCentered("Name Instructions:");
    WriteLineCentered("- English letters only.");
    WriteLineCentered("- No spaces or numbers allowed.");
    WriteLineCentered("- Must be at least 3 letters.");
    Console.WriteLine();
```



The application provides essential CRUD operations to manage contacts efficiently.

- Create: Add new contacts.
- Read: Display and view contact details.
- Update: Modify existing contact information.
- Delete: Move contacts to Trash using Soft Delete.

Key Concept: CRUD Operations + LINQ + Entity Framework Core.

SEARCH & ORGANIZE CONTACTS

The application provides essential CRUD operations to manage contacts efficiently.

- The Contact Book provides powerful tools to find, organize, and view contacts easily.

- 1. Search Contacts
- Search by name, phone number, email, category, or address.
- Display matching contacts only.
- Exclude deleted contacts from search results.

- 2. Organize by Categories
- Family, Work, Friends and Favorite

- 3. View Contact Details
- Display the selected contact's information.
- Show name, category, phone, email, address, and creation date.

```
private void SearchContacts()
{
    Console.ForegroundColor = ConsoleColor.Blue;
    WriteLineCentered("--- Search contacts ---");
    var term = Ask("Search (name, phone, email, category or address)", required: true, cu
    if (term == "BACK_SIGNAL") { Console.ResetColor(); return; }
    term = term.ToLower();

    var results = _db.Contacts
        .AsNoTracking()
        .Where(c => !c.IsDeleted && (c.FirstName.ToLower().Contains(term)
                    || c.LastName.ToLower().Contains(term)
                    || (c.FirstName.ToLower() + " " + c.LastName.ToLower()).Contains(term)
                    || c.PhoneNumber.Contains(term)
                    || c.Email.ToLower().Contains(term)
                    || c.Category.ToLower().Contains(term)
                    || (c.Address != null && c.Address.ToLower().Contains(term))))
        .OrderBy(c => c.Id)
        .ToList();

    Console.WriteLine();
    PrintTable(results, ConsoleColor.Blue);
    Console.ResetColor();
}
```

TRASH MANAGEMENT & CONTACT RECOVERY

The Contact Book allows users to manage deleted contacts, restore them, or permanently remove them.

1. View Trash
   - Display all deleted contacts.
   - Show contacts marked as deleted.
   - Keep deleted contacts separate from active contacts.

2. Restore Contact
   - Select a contact from the Trash.
   - Restore it to the active contacts list.
   - Clear its deletion status and deletion date.

3. Empty Trash
   - Permanently delete all contacts in the Trash.
   - Ask for confirmation before deletion.

```
private void ViewTrash()
{
    Console.ForegroundColor = ConsoleColor.Green;
    WriteLineCentered("--- Trash (Deleted Contacts) ---");
    Console.ResetColor();

    var deletedContacts = _db.Contacts
        .AsNoTracking()
        .Where(c => c.IsDeleted)
        .OrderBy(c => c.Id)
        .ToList();

    PrintTable(deletedContacts, ConsoleColor.Green);
}
```

```
private void RestoreContact()
{
    Console.ForegroundColor = ConsoleColor.Green;
    WriteLineCentered("--- Restore contact from Trash ---");

    List<Contact> deletedContacts = _db.Contacts.Where(c => c.IsDeleted).OrderBy(c => c.Id).
    if (deletedContacts.Count == 0)
    {
        WriteLineCentered("Trash is empty.");
        Console.ResetColor();
        return;
    }

    Console.WriteLine();
    PrintTable(deletedContacts, ConsoleColor.Green);
    WriteCentered("Enter the display index (Id) of the contact to restore: ");

    var input = ReadLine().Trim();
    if (input.Equals("b", StringComparison.OrdinalIgnoreCase)) { Console.ResetColor(); retur
```

DATA VALIDATION & ERROR HANDLING

The Contact Book validates user input and handles database errors to keep contact data accurate and reliable.

1. Name Validation
   - English letters only.
   - No spaces or numbers.
   - Minimum 3 characters.

2. Phone Number Validation
   - Supports multiple country codes.
   - Checks the required number of digits.
   - Validates Egyptian mobile prefixes.
   - Rejects unrealistic repeated digits.

3. Email Validation
   - Supports Gmail and Yahoo only.
   - Checks the email format.
   - Allows email to be optional.

```
private static string? ValidateName(string value)
{
    if (!Regex.IsMatch(value, @"^[a-zA-Z]+$"))
    {
        PrintWarning("Invalid name format! - Must use English letters only
        return "Please re-enter a valid name.";
    }

    if (value.Length < 3)
    {
        PrintWarning("Invalid name format! - Must be at least 3 letters.",
        return "Please re-enter a valid name.";
    }

    return null;
```

```
private bool Save(Action? change = null)
{
    try
    {
        change?.Invoke();
        _db.SaveChanges();
        return true;
    }
    catch (DbUpdateException ex)
    {
        PrintWarning("Could not save changes to the database.",
        WriteLineCentered(ex.InnerException?.Message ?? ex.Mess
        _db.ChangeTracker.Clear();
        return false;
    }
```

```
private static bool IsRealisticPhoneNumber(string rawNumber)
{
    if (new string(rawNumber[0], rawNumber.Length) == rawNumber)
    {
        return false;
    }

    int consecutiveCount = 1;
    for (int i = 0; i < rawNumber.Length - 1; i++)
    {
        if (rawNumber[i] == rawNumber[i + 1])
        {
            consecutiveCount++;
            if (consecutiveCount > 3)
            {
                return false;
            }
        }
        else
        {
            consecutiveCount = 1;
        }
    }
```

PERMANENT DELETION & DATABASE MANAGEMENT

Permanent Deletion allows users to completely remove contacts from the database instead of just moving them to the Trash.

Key Features

- Delete Single Contact: Permanently removes a selected contact.
- Empty Trash: Permanently deletes all contacts currently in the Trash.
- Delete All Contacts: Removes every contact from the database.
- Confirmation Before Deletion: Requires the user to confirm destructive actions.
- Database Cleanup: Resets the SQLite auto-increment sequence after deleting all contacts.

```
private void DeleteContactPermanently()
{
    Console.ForegroundColor = ConsoleColor.Red;
    WriteLineCentered("--- Delete contact permanently ---");

    var contact = FindContact(track: true, includeDeleted: true, ConsoleColor.Red);
    if (contact is null) { Console.ResetColor(); return; }

    WriteLineCentered($"You are about to permanently delete: {FullName(contact)} ({contact.PhoneNumber})");
    WriteCentered("Are you sure? (Y/n): ");
    var answer = ReadLine().Trim();

    if (!answer.Equals("y", StringComparison.OrdinalIgnoreCase))
    {
        PrintCancelled();
        Console.ResetColor();
        return;
    }

    if (Save(() => _db.Contacts.Remove(contact)))
    {
        WriteLineCentered("Contact deleted permanently.");
    }
    Console.ResetColor();
}
```

```
private void DeleteAllContacts()
{
    Console.ForegroundColor = ConsoleColor.Red;
    WriteLineCentered("--- Delete ALL contacts (Permanent) ---");
    WriteCentered("Are you sure you want to delete ALL contacts permanently? (Y/n): ");
    var answer = ReadLine().Trim();

    if (!answer.Equals("y", StringComparison.OrdinalIgnoreCase))
    {
        PrintCancelled();
        Console.ResetColor();
        return;
    }
```

MAIN MENU & APPLICATION NAVIGATION

The Contact Book application uses a console-based menu system that allows users to access all features through numbered options.

Key Features

- 15 Operations: Add, view, search, update, organize, and delete contacts.
- Category Navigation: Quickly access Family, Work, Friends, and Favorites.
- Trash Management: View, restore, or permanently delete contacts.
- Input Handling: The program checks the selected option and handles invalid choices.
- Continuous Execution: The menu appears again after each operation until the user chooses Exit.

```
while (true)
{
    ShowMenu();
    var choice = ReadLine().Trim();
    Console.WriteLine();

    switch (choice)
    {
        case "1": AddContact(); break;
        case "2": ListContacts(); break;
        case "3": ViewContact(); break;
        case "4": ListCategory("Family"); break;
        case "5": ListCategory("Work"); break;
        case "6": ListCategory("Friends"); break;
        case "7": ListCategory("Favorite"); break;
        case "8": SearchContacts(); break;
        case "9": UpdateContact(); break;
        case "10": DeleteContact(); break;
        case "11": ViewTrash(); break;
        case "12": RestoreContact(); break;
        case "13": EmptyTrash(); break;
        case "14": DeleteAllContacts(); break;
        case "15": DeleteContactPermanently(); break;
        case "0":
            Console.ForegroundColor = ConsoleColor.DarkRed;
            WriteLineCentered("Goodbye!");
```

CONCLUSION

The Contact Book application provides an organized way to manage contacts through a console-based interface.

Main Achievements:

- Implemented full CRUD operations.
- Added contact search and category filtering.
- Supported Trash, Restore, and Permanent Deletion.
- Applied input validation and error handling.
- Used Entity Framework Core with a database to store contact information.

THANK YOU

GitHub Repo